Лабораторна робота 3

ОСНОВИ, ФУНКЦІОНАЛЬНЕ ТА ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ МОВОЮ JAVASCRIPT

Мета – вивчити типи даних, основні оператори, умовні конструкції, цикли, типи функцій мови програмування JS, набути навиків та вмінь у створенні програмного коду.

Теоретичні відомості

JavaScript (JS) — це мова програмування, яка з'явилася для додавання динамічності веб-сторінкам. Однак сучасний JS використовується набагато ширше. JavaScript було створено *Брендоном Айком* в 1995 в Netscape Communications. JS є реалізацією стандарту мови програмування *ECMAScript.* JS є скриптовою мовою, що обробляється інтерпретатором окремими інструкціями (командами). Кожна команда JS зазвичай пишеться в окремому рядку і в більшості випадків завершується крапкою з комою.

JavaScript - реєстрозалежна мова програмування, тобто JS розрізняє великі і малі літери. Код JavaScript повинен визначатися за допомогою кодування Unicode.

У кожній мові програмування існують набори домовленостей, що регламентують особливості оформлення коду, порядок використання синтаксичних конструкцій, що допускають неоднозначності, тощо. Це дозволяє зробити код більш зрозумілим. Ці домовленості визначають стиль написання коду. Мова JavaScript не є винятком.

Для оформлення коду мовою JS, використовують наступні правила:

• camelCase для іменування об'єктів, функцій та змінних.

• PascalCase для іменування конструкторів класів.

• підкреслення _ як префікс для іменування внутрішніх методів та змінних об'єктів

1. Змінні. Константи. Типи даних:

Змінна – це комірка пам'яті (контейнер), яка використовується для зберігання значень даних. Для оголошення змінної використовується ключове слово «let» або «var», основна відмінність між якими полягає у тому, що let створює змінну з обмеженою областю видимості і доступ до неї можна з обмежених фрагментів коду, що підвищує його надійність. Детальніше про відмінності між ними у [14].

Для того, щоб визначити **константу** – треба замість ключового слова let використовувати const. Значення константи не можна змінити. При спробі змінити значення константи отримаєте помилку. Константа має бути обов'язково ініціалізована відразу при її оголошенні.

Типи даних JavaScript можна розділити на дві групи: елементарні (або примітивні) та об'єктні.

Перелік елементарних даних у JS:

• Boolean використовується для зберігання двох логічних значень: *«істина»* або *«хиба»*. Часто застосовується для умов розгалуження у коді програми;

• Number дозволяє зберігати цілі числа та числа з рухомою крапкою в діапазоні від $-(2^{53}-1)$ до $(2^{53}-1)$;

• Infinity, -Infinity та NaN (Not-A-Number);

• BigInt дозволяє працювати з цілими числами довільного розміру. Щоб вказати використання типу BigInt, необхідно в кінці додати літерал “n”; • Symbol використовується для генерації унікальних значень, які будуть ідентифікаторами змінних інших типів даних (наприклад, об'єктів); • String використовується для зберігання рядків. Рядки JavaScript повинні бути укладені в одинарні або подвійні лапки;

• Null свідчить, що отримано порожнє значення. Така ситуація може бути коли ми намагаємося отримати об'єкт через API взаємодії з іншими сервісами, але цей об'єкт не існує. Літерал null може також використовуватися для спеціального присвоєння порожнього значення;

• Undefined вказує, що значення змінної не визначено.

Object JavaScript — це набір властивостей та методів. Об'єкти доцільно використовувати для групування даних та роботи з ними. Об'єкти також можуть зберігати методи (функції) для роботи з даними. Отримувати значення властивостей об'єкта можна через крапку або квадратні дужки.

Наведемо приклад для зберігання інформації про користувача (індентифікатор, ім’я, прізвище та вік) та виведемо у консоль його ідентифікатор, ім’я та прізвище:

```
let id = Symbol("id");
let user = {
 [id]: 1,
 firstName: 'Stepan',
 secondName: 'Kravchuk',
 age: 20
};
console.log(user.firstName);
console.log(user["secondName"]);
console.log(user[id]);
```

2. Оператори. Динамічна типізація. Неявне перетворення типів: Для визначення типу змінної використовується оператор string typeof operand, який повертає тип змінної у вигляді рядка. Зверніть увагу, що для null оператор typeof поверне тип "object". Ця особливість була пов'язана з тим, що очікувався об'єкт, а отримано значення null.

Унарний оператор застосовується до одного операнда. Унарні оператори

інкрементів — збільшення змінної на одиницю та декремент — зменшення змінної на одиницю поділяються на префіксні та постфіксні.

2

Операції інкремент та декремент застосовуються лише до змінних. Ці операції не застосовуються до числових літералів.

Наприклад, оператор унарний мінус "-" змінить знак змінної на протилежний.

Бінарний оператор застосовується до двох операндів. Наприклад, оператор мінус може бути як унарним тобто застосовується до одного операнда (робить число негативним), так і бінарним (операція віднімання).

За призначенням оператори поділяються на арифметичні, логічні, оператори порівняння, присвоєння та інші. У Таблиці 14 набедено перелік бінарних операторів, їх опис та приклад коду.

Таблиця 14. Перелік бінарних операторів та їх опис

Оператор	**Опис**	**Приклад коду**

+	арифметичний оператор додавання чисел, операція конкатенації, оператор явного перетворення	console.log (5 + 2);//7
		let result ='Арифметичні ' + 'оператори';
		console.log (result);//”Арифметичні оператори”

-	арифметичний оператор віднімання і застосовується лише до чисел	console.log (5 - 2); // 3
*	арифметичний оператор, здійснює множення двох операндів і застосовується тільки до чисел	console.log (5 * 2); //10
**	арифметичний оператор піднесення до степеня. Лівий операнд підносимо у степінь правого	console.log (5 ** 2); //25
/	арифметичний оператор, дає результат частки від ділення лівого операнда на правий і застосовується тільки до чисел	console.log(5 / 2); //2.5

%	арифметичний оператор остача від ділення (ділення за модулем) повертає все, що залишилося від цілочисельного ділення лівого операнда на правий і застосовується тільки до чисел	console.log(5 % 2); //1
=	оператор присвоєння. Спочатку обчислюється значення виразу у правій частині (після знака "="), потім це значення присвоюється змінній, яка розміщується у лівій частині	
+=	x+=y рівносильно x=x+y	
-=	x-=y рівносильно x=x-y	
*=	x*=y рівносильно x=x*y	
/=	x/=y рівносильно x=x/y	

%=	x%=y рівносильно x=x%y	

У JS, як і інших мовах програмування, оператори порівняння повертають булевий тип даних. Тобто результатом завжди буде true чи false. Перелік операторів порівняння надано у Таблиці 15. При порівнянні завжди відбувається перетворення даних до числа. Крім порівняння рядків та суворої рівності (нерівності).

Таблиця 15. Перелік операторів порівняння

Оператор	**Приклад**	**Повертає значення true, якщо**
>	a>b	a більше b
>=	a>=b	a більше або дорівнює b
<	a<b	a менше b
<=	a<=b	a менше або дорівнює b
==	a==b	a дорівнює b

===	a===b	a дорівнює b (без приведення типів)
!==	a!==b	a не дорівнює b чи їх типи відрізняються

При порівнянні у JavaScript деякі типи даних не перетворюються до числового типу. Так, значення undefined незрівнянно з іншими значеннями:

```
console.log( undefined > 0 ); // false
console.log( undefined < 0 ); // false
console.log( undefined == 0 ); // false
```

При застосуванні оператора порівняння "==" до null або undefined, чисельне перетворення не відбувається, оскільки null може дорівнювати лише null або undefined, і нічому іншому. NaN не дорівнює нічому, навіть самому собі. NaN !== NaN

У JS існує три логічні оператори: || (АБО), && (І) та ! (НЕ). На відміну від строго типізованих мов програмування, дані оператори можуть застосовуватися до значень будь-яких типів і результат теж може бути будь-яким типом.

Перетворення типів — це процес перетворення значення з одного типу на інший. Будь-який тип даних, будь то примітивний тип або об'єкт, може бути перетворений на інший тип.

Для явного перетворення до булевого значення потрібно застосувати функцію Boolean() або подвійне заперечення !!.

Неявне перетворення відбувається у логічному контексті if (val) { … } або за умови застосування логічних операторів (|| && !).

Для явного перетворення до числа потрібно застосувати функцію Number(). Неявне перетворення дещо заплутаніше, оскільки воно викликається у більшій кількості випадків, коли застосовуються:

• оператори порівняння (>, <, <=, >=);

• бінарні оператори (| & ^ ~);

• арифметичні оператори (- + * / %). Зверніть увагу, що бінарний оператор + не викликає чисельного перетворення, якщо один із операндів є рядком; • унарний оператор +;

• оператор нестрогої рівності == (включаючи !=). Зверніть увагу, що цей оператор не викликає чисельного перетворення, якщо обидва операнди є рядками.

Для явного приведення значення рядка необхідно застосувати до нього функцію String(). Неявне перетворення буде викликане бінарним оператором +, коли один із операндів є рядком.

3. Умовні конструкції. Цикли:

Набір згрупованих команд називається **блоком коду**. Блоки використовуються для обмеження області видимості змінних та функцій, а

також дозволяють звертатися до блоку операторів, як до єдиної інструкції. Блоки можуть бути вкладені один в одного або можуть бути порожніми.

У JS блок коду позначається фігурними дужками {}. Блок зазвичай використовується з конструкціями if, for, while, function та ін. Починаючи з ES6, блоки коду обмежують область видимості змінних. Важливими є умовні конструкції, наведемо їх перелік та синтаксис у Таблиці 16.

Таблиця 16. Перелік умовних контрукції та їх синтаксис

Назва	**Синтаксис**	**Приклад коду**

if – оператор розгалуження	if (умова) {
	//виконуються оператори, якщо умова істинна
	}
		let weather = 'дощ';
		if (weather==='дощ'){
		console. log('Взяти парасольку');
		}

if–else	if (умова) {
	// виконуються оператори, якщо умова є істинною
	} else {
	//виконуються оператори, якщо умова хибна
	}
		if (a%2===0) {
		console.log('Число парне');
		} else {
		console.log('Число непарне');
		}

тернарний (потрійний) оператор	умова ? вираз1 : вираз2	a%2==0 ? console.log('Число парне') : console.log('Число непарне');

switch	switch (змінна) {
	case значення1: //оператори, якщо змінна дорівнює значення1
	//[break;]
	case значення2: //оператори, якщо змінна дорівнює значення2
		let distance = 10;
		let transport ="літак";
		switch(transport){
		case 'велосипед' :
		console.log(`Час у дорозі - ${distance/10}`);
		break;

	// [break;] ... case значенняN: //оператори, якщо змінна дорівнює значенняN //[break;] default: //оператори, якщо не знайдено відповідного case //[break;] }	case 'авто' : console.log(`Час у дорозі - ${distance/80}`); break; case 'потяг' : console.log(`Час у дорозі - ${distance/100}`); break; case 'літак' : console.log(`Час у дорозі - ${distance/500}`); break; default: console.log('Про такий траспорт не знаю');} // Час у дорозі - 0.02

В оператор «if–else» допускається вкладати інші оператори «if» або «if–else». Такі ланцюжки використовуються для комбінування else та if, щоб перевірити різні взаємовиключні варіанти.

При створенні ланцюжка операторів if–else–if–else діють *наступні правила:* • оператор if може не мати або мати лише один else, який стоїть після будь якого else–if;

• оператор if може не мати або мати багато else–if і вони повинні стояти до його else;

• після того, як будь-який else–if виконаний (тобто результат умови true), то жодний з else–if або else не перевірятимуться.

Синтаксис ланцюжка операторів if–else–if–else наступний:

if (умова 1) { //оператори } else if (умова 2) { //оператори} else if (умова 3) { //оператори } else { //оператори}

В JS в операторі if як умову можна передавати будь-який вираз або літеральну константу.

У JS існують наступні оператори циклу:

• for або цикл із лічильником найчастіше використовується тоді, коли заздалегідь відомо, скільки разів потрібно виконати блок коду. Для створення такого циклу використовується оператор for, який складається з трьох блоків у круглих дужках, розділених “;” та тіла циклу, укладеного у фігурні дужки. Синтаксис:

for (/*блок ініціалізації*/;(/*блок умови виходу*/;(/*блок змінної цикла*/;) {/*оператори*/};

• while або цикл з передумовою доцільно в тих випадках, коли потрібно повторювати частину дій, а кількість повторень невідома і залежить від певної умови, наприклад, від даних, які вводить користувач. Синтаксис:

```
while (Умова) {/*оператори*/};
```

• do-while або цикл з постумовою доцільно використовувати у випадках що й з передумовою, проте його тіло завжди виконається щонайменше один раз. Синтаксис:

```
do {/*оператори*/} while (умова);
```

Оператори переходу — це оператори, які дозволяють здійснити перехід з точки, де оператор викликається, в іншу частину програми. Оператори break і continue. Основна відмінність break від continue полягає в тому, що break перериває виконання всієї конструкції, а continue лише поточної ітерації.

4. Масиви. Функції. Область видимості:

Масиви призначені для роботи з даними. Для створення масиву застосовується вираз new Array(). Також існує більш короткий спосіб оголошення масиву: let myArray = []. У цьому випадку створюється порожній масив. Внесемо дані в нього:

```
let people = ["Mykola", "Yuriy", "Andriy"];
```

Для звернення до окремих елементів масиву використовуються індекси. Відлік починається з нуля, тобто перший елемент має індекс 0, а останній – 2:

```
console.log(people[0]); // Mykola
```

На відміну від інших мов програмування, в JS масиви не строго типізовані, тобто один масив може зберігати дані різних типів:

```
let objs = ["Andriy ", 44, 10.79, true];
```

Масиви можуть бути одновимірними та багатовимірними. Щоб дізнатись довжину масиву, використовують властивість length Розглянемо приклад двовимірного масиву:

```
var people = [
 ["Mykola ", 33, false],
 ["Andriy", 44, true],
 ["Yuzik", 55, false]
];
console.log(people[0]); // ["Mykola ", 33, false]
console.log(people[1]); // ["Andriy ", 44, true]
console.log(people[0][1]); // 33
```

У Таблиці 17 наведено основні методи для роботи з масивами та їх

короткий опис. Повний перелік у [14].

Таблиця 17. Методи для роботи з масивами та їх опис

Метод	**Опис**

push()	додає елемент в кінець масиву;
pop()	видаляє останній елемент масиву;
shift()	повертає і видаляє перший елемент масиву;
concat()	повертає новий масив, утворений об’єднанням довільної кількості масивів та окремих значень, які перераховані в методі як аргументи;
join()	об’єднує всі елементи масиву в один рядок;
unshift()	додає новий елемент в початок масиву;
splice()	видаляє всі елементи масиву, починаючи із заданого індексу;
sort()	сортує масив;
every()	перевіряє, чи всі елементи відповідають заданій умові;
reverse()	змінює порядок розташування елементів масиву на обернений;
indexOf()	повертає індекс першого входження шуканого елемента в масиві; в протилежному випадку повертає - 1;
lastIndexOf()	повертає індекс останнього входження шуканого елемента в масиві; в протилежному випадку повертає - 1;
filter()	приймає функцію умови й повертає масив тих елементів, які відповідають вказаній умові;
some()	перевіряє, чи відповідає хоча б один елемент умові, і повертає true; якщо елементів, що відповідають умові, в масиві немає, то метод повертає false;

forEach()	екземпляри виконують надану функцію один раз для кожного елемента масиву
map()	екземпляри створюють новий масив, заповнений результатами виклику наданої функції для кожного елемента у викликаючому масиві

Функція в JS — це тип об'єктів, який за допомогою засобів мови дозволяє формалізувати поведінку та обробку даних. Функції мають такі особливості: • функція — це підпрограма, призначена для виконання конкретного завдання;

• функція виконується під час виклику;

• у функцію можуть бути передані значення, які використовуються всередині неї;

• функція завжди повертає значення. Якщо значення, що повертається, не вказано, функція поверне undefined.

Функція JS визначається за допомогою ключового слова function, після якого йде ім’я та дужки (). Назви функцій можуть містити літери, цифри, підкреслення та знаки долара (за тими самими правилами, що й змінні). У дужках можуть бути назви параметрів, розділені комами. Усередині функції аргументи

(параметри) поводяться як локальні змінні. Код, який виконується функцією, розміщується у {}.

Для створення функції, її необхідно оголосити. Оголошення функцій бувають таких видів:

• function declaration: function name (parameters) { };

• function expression: let name = function (parameters) { };

• стрілкова функція, детальніше у [14]: () => expression

Стрілковий функціональний вираз — це компактна альтернатива традиційному функціональному виразу з деякими семантичними відмінностями та навмисними обмеженнями у використанні:

• не мають власних прив'язок до this, аргументів або super, і не повинні використовуватися як методи;

• не можна використовувати як конструктори. Їх виклик з new викликає помилку TypeError. Вони також не мають доступу до ключового слова new.target; • не можуть бути створені як функції-генератори.

Зауважимо також, що *кількість параметрів* під час опису функції та виклику може відрізнятися. Якщо під час виклику функції передано менше параметрів, ніж є в описі, непередані параметри дорівнюватимуть undefined.

Якщо змінну оголошено всередині функції, до неї не можна отримати доступ зовні. Виходячи з цього, якщо потрібна змінна тільки для функції, то її потрібно оголошувати в області видимості функції (scope).

Функція матиме доступ до всіх змінних та функцій, оголошених у зовнішньому scope.

Функція, оголошена у глобальному scope, має доступ до всіх змінних у глобальному scope. Функція, оголошена всередині іншої функції, ще має доступ до всіх змінних її батьківської функції та інших змінних, до яких ця батьківська функція має доступ.

Зазначимо, що для Function declaration функція може бути викликана до опису. Цей механізм отримав назву hoisting (піднімання). Його суть полягає в тому, що перед виконанням коду інтерпретатором відбувається певна оптимізація та опис функції піднімається перед її викликом. Якщо бути більш точними, він розміщується на початку області видимості. Функція, описана як Function expression, не піднімається.

У деяких випадках функцію необхідно викликати одразу. Для такого завдання використовується наступний синтаксис:

(function (){ код})();

5. Об'єктно-орієнтований js-код:

Використовується у випадку, коли потрібно описати сутність «людина», яка має свої характеристики: ім'я, вік, стать і так далі. Тоді «людина» буде виступати як складна комплексна структура, що складається з окремих властивостей: вік, зріст, ім'я, прізвище і т.д.

Для роботи з подібними структурами в JS використовуються об'єкти. Кожен об'єкт може зберігати властивості, які описують його стан, і методи, які

описують його поведінку. Створення нового об'єкта: let person = new Object(); або let person = {}.

Після створення об'єкта можна визначити в ньому властивості. Щоб визначити властивість, потрібно після імені об'єкта через крапку вказати ім'я властивості і присвоїти їй значення. Також для присвоєння значення властивості використовується символ двокрапки, а після визначення значення ставиться кома (а не крапка з комою).

Методи об'єкта визначають його поведінку, тобто дії, які він виконує або які виконується з ним. Фактично методи являють собою функції. Наведемо приклад рядків коду для виводу ім'я і віку людини:

```
let person = {};
person.name = "Mykola";
person.age = 16;
person.display = function() {
 console.log(person.name);
 console.log(person.age);
};
person.display();// виклик методу
```

Щоб звернутися до властивостей або методів об’єкта всередині цього об’єкта, використовується ключове слово **this**, яке означає посилання на поточний об’єкт.

Існує також альтернативний спосіб визначення властивостей і методів за допомогою синтаксису масивів:

```
let person = {};
person ["name"] = " Mykola ";
person ["age"] = 26;
person ["display"] = function() {
 console.log(person.name);
 console.log(person.age);
};
person ["display"](); //виклик методу
person.display(); //виклик методу спрацює так само
```

Можна видаляти властивості і методи за допомогою оператора delete. Як і у випадку з їх визначенням, можна видаляти властивості двома способами: • delete об’єкт.властивість;

• delete об’єкт["властивість"].

Після видалення властивість буде не визначена, тому при спробі звернення до неї, повертається значення undefined.

Оскільки об’єкти відносяться до типу object, а значить, мають всі його методи і властивості, то, відповідно, також можуть використовувати метод hasOwnProperty(), який визначений в типі object. Метод повертає true у випадку, якщо об’єкт має не успадковану властивість із зазначеним ім’ям та false, якщо

об’єкт не має такої властивості або ця властивість успадкована від свого об’єкта прототипу.

Крім створення нових об’єктів, JavaScript надає можливість створювати нові типи об’єктів за допомогою конструкторів. *Конструктор* – це функція, яка призначена для визначення та задання властивостей та методів. Для звертання до властивостей і методів використовується ключове слово this. Оператор instanceof () дозволяє перевірити, за допомогою якого конструктора створено об’єкт. Якщо об’єкт створений за допомогою певного конструктора, то оператор повертає true.

Інкапсуляція є одним із ключових понять об’єктно-орієнтованого програмування і полягає в прихованні стану об’єкта від прямого доступу ззовні. За замовчуванням, всі властивості об’єктів публічні, тобто загальнодоступні, і до них можна звернутися з будь-якого місця програми. Для оголошення локальних змінних в конструкторах використовують знак підкреслення у назві.

JS підтримує успадкування. Це дозволяє при створенні нових типів об’єктів

за необхідності успадкувати їх функціонал від уже існуючих. Існує спосіб визначення об’єктів за допомогою класів. Клас являє собою опис об’єкта, його стану і поведінки, а об’єкт є конкретним екземпляром класу. Для визначення класу використовується ключове слово class:

class Person { }.

Після слова class вказується назва класу (в даному випадку, клас називається Person), потім у фігурних дужках визначається тіло класу. Можна визначити й анонімний клас і присвоїти його змінній:

let Person = class {}.

Після цього можна створити об’єкти класу за допомогою конструктора:

```
let andriy = new Person();
let ivan = new Person();
```

Для створення об’єкта за допомогою конструктора використовується ключове слово new, після якого виклик конструктора, фактично, виклик функції по імені класу. За замовчуванням, класи мають один конструктор без параметрів. Викликаючи такий конструктор, в нього не передається жодних параметрів. Також можна визначити в класі свої конструктори, властивості і методи. Конструктор визначається за допомогою методу з іменем constructor. По суті, це звичайний метод, який може мати параметри. Основна мета конструктора – ініціалізувати об’єкт початковими даними.

Одні класи можуть успадковуватися від інших. Спадкування дозволяє скоротити обсяг коду в класах-нащадках. Для успадкування одного класу від іншого у визначенні класу застосовується ключове слово **extends**, після якого вказується назва базового класу.

Наведемо приклад коду, де клас Employee успадковується від класу Person:

```
class Person {
 constructor(n, a) {
```



```
 this.name = n;
 this.age = a;
 }
 print() {
 console.log(this.name, this.age);
 }
}
class Employee extends Person {
 constructor(n, a, c) {
 super(n, a);
 this.company = c;
 }
 print() {
 super.print();
```

```
 console.log("Працює у", this.company);
 }
 work() {
 console.log(this.name, " є працьовитим");
 }
}
let andriy = new Person("Andriy", 34);
let ivan = new Employee("Ivan", 36, "Google");
andriy.print(); // Andriy 34
ivan.print(); // Ivan 36 Працює у Google
ivan.work(); // Ivan є працьовитим
```

Статичні методи викликаються для всього класу в цілому, а не для окремого об'єкту. Для їх визначення застосовується оператор static.

6. Об'єкти Date, Math, Number та String:

Об'єкт **Date** дозволяє працювати з датами і часом в JS. снують різні способи створення об'єкта Date.

Перший спосіб полягає у використанні порожнього конструктора без параметрів:

```
let currentDate = new Date();
```

У цьому випадку, об'єкт буде містити поточну дату комп'ютера. Другий спосіб полягає у передачі в конструктор Date кількості мілісекунд, які пройшли з початку епохи Unix, тобто з 1 січня 1970 року 00:00:00 GMT:

```
let myDate = new Date(1359270000000);
```

Третій спосіб полягає в передачі в конструктор Date дня, місяця і року дати:

```
let myDate1= new Date("27 March 2008");
// або так
let myDate2 = new Date("3/27/2008");
```

Якщо використовується повна назва місяця, то вона пишеться англійською; якщо використовується скорочений варіант, то формат "місяць/день/рік". Четвертий спосіб полягає в передачі в конструктор Date всіх параметрів дати і часу:


```
 // Tue Dec 25 2012 18:30:20 GMT + 0300 (RTZ 2 (зима))
let myDate = new Date(2012, 11, 25, 18, 30, 20, 10);
```

У цьому випадку, в зазначеному порядку використовуються наступні параметри:

new Date (рік, місяць, число, час, хвилини, секунди, мілісекунди).

При цьому потрібно враховувати, що відлік місяців починається з нуля, тобто січень – 0, а грудень – 11.

Для отримання різних складових дати використовуються методи:

• getDate() повертає день місяця;

• getDay() повертає день тижня (відлік починається з нуля – неділя,

останній день – 6 – субота);

• getMonth() повертає номер місяця (відлік починається з нуля, тобто місяць з номером 0 – січень);

• getFullYear() повертає рік;

• toDateString() повертає повну дату у вигляді рядка;

• getHours() повертає годину (від 0 до 23);

• getMinutes() повертає хвилини (від 0 до 59);

• getSeconds() повертає секунди (від 0 до 59);

• getMilliseconds() повертає мілісекунди (від 0 до 999);

• toTimeString() повертає повний час у вигляді рядка.

Для задання дати використовують такі методи об'єкта Date:

• setDate() задає день дати;

• setMonth() задає місяць (відлік починається з нуля, тобто місяць з номером 0 - січень);

• setFullYear() задає рік;

• setHours() задає годину;

• setMinutes() задає кількість хвилин;

• setSeconds() задає кількість секунд;

• setMilliseconds()задає кількість мілісекунд.

Об'єкт **Math** надає набір математичних функцій, які можна використовувати при обчисленнях. Розглянемо основні з них: • abs() повертає абсолютне значення числа;

• min() і max() повертають, відповідно, мінімальне та максимальне значення з набору чисел (ці функції не обов'язково повинні приймати два числа, в них можна передавати більшу кількість чисел);

• ceil() округлює число до найближчого більшого цілого числа (фактично, повертає цілу частину числа +1);

• floor() округлює число до найближчого меншого цілого числа (фактично, повертає цілу частину числа);

• round() заокруглює число до найближчого меншого цілого числа, якщо десятих менше 5; якщо ж дорівнює або більше 5, то округлення до найближчого більшого цілого числа;

• random() повертає випадкове число з плаваючою крапкою з діапазону від 0 до 1;

• pow() повертає число, піднесене до степеня;

• sqrt() повертає квадратний корінь числа;

• log() повертає натуральний (за основою e=2,718) логарифм числа; • тригонометричні функції sin(), cos(), tan() повертають, відповідно, синус, косинус, тангенс кута, що задається параметром у радіанах;

• обернені тригонометричні функції: asin() обчислює арксинус числа, acos() - арккосинус, а atan() - арктангенс.

Крім методів, об’єкт Math, також визначає набір вбудованих констант, які можна використовувати в різних обчисленнях, зокрема:

• Math.PI (число PI): 3.141592653589793;

• Math.E (число e або число Ейлера): 2.718281828459045

Об’єкт **Number** представляє число. Щоб створити число, потрібно передати в конструктор Number число або рядок, що містить число.

Об’єкт Number надає ряд властивостей і методів. Деякі його властивості: • Number.MAX_VALUE: найбільше можливе число, приблизно 1.79E + 308, числа, які більше цього значення, розглядаються як Infinity;

• Number.MIN_VALUE: найменше можливе додатнє число, приблизно 5e 324;

• Number.NaN: спеціальне значення, яке вказує, що об’єкт не є числом; • Number.NEGATIVE_INFINITY: значення, яке позначає від’ємну нескінченність (-Infinity) і яке виникає при переповненні, наприклад, при додаванні двох від’ємних чисел, які за модулем рівні number.MAX_VALUE; • Number.POSITIVE_INFINITY: додатня нескінченність (Infinity), також, як і від’ємна, виникає при переповненні, тільки тепер в додатному напрямку. Деякі основні методи об’єкту Number :

• isNaN() визначає, чи є об’єкт числом, якщо об’єкт не є числом, то повертає значення true;

• parseFloat() перетворює рядок в число з плаваючою крапкою; • toFixed() залишає в числі з плаваючою точкою певну кількість знаків у дробовій частині.

Для роботи з рядками призначений об’єкт **String** і конструктор String:

```
let name = new String("George");
```

Об’єкт String має великий набір властивостей і методів, за допомогою яких можна працювати з рядками. Зокрема, властивість length повертає довжину рядка.

Метод repeat() дозволяє створити рядок шляхом багаторазового повторення іншого рядка, приймає як аргумент кількість повторів, причому повтори не розділяються жодними символами (console.log(hello.repeat(2));)

Для пошуку в рядку деякого підрядка використовуються методи indexOf() (індекс першого входження підрядка) і lastIndexOf() (індекс останнього входження підрядка). Ці методи мають два параметри:

• підрядок, який потрібно знайти;

• необов’язковий параметр, який вказує, з якого символу виконувати пошук підрядка в рядку.

Обидва ці методи повертають індекс символу, з якого в рядок починається підрядок. Якщо підрядок не знайдено, то повертається число «-1». Ще один метод includes() повертає true, якщо рядок містить певний підрядок.

Для того, щоб отримати (скопіювати) з рядка підрядок, застосовуються методи substr() і substring().

Метод substring() дозволяє отримати (скопіювати) символи з рядка

(підрядок) між двома заданими індексами, які передаються як параметр методу, або від певного індексу до кінця рядка, якщо другий параметр не вказано.

Метод substr() також дозволяє отримати (скопіювати) з рядка певну кількість символів (підрядок), починаючи із заданого індексу. Цей метод приймає як перший параметр початковий індекс підрядка, а як другий – довжину підрядка, який потрібно отримати. Якщо другий параметр не вказано, то копіюється решта рядка.

Для зміни регістру призначені методи toLowerCase() (перетворення в нижній регістр) і toUpperCase() (перетворення у верхній регістр). Щоб отримати певний символ в рядку за індексом, застосовують методи charAt() і charCodeAt(). Обидва методи як параметр приймають індекс символу: але як результат метод charAt() повертає сам символ, а метод charCodeAt() – числовий код цього символу.

Метод concat() об'єднує два рядки. Метод replace() замінює перше входження одного підрядка на інший. Метод split() розбиває рядок на масив підрядків за певним роздільником. Як роздільник використовується параметр, що передається в метод.

Завдання

1. Вивчити типи даних, основні оператори, умовні конструкції, цикли, типи функцій мови програмування JavaScript та основи об'єктно-орієнтованого підходу в ній;

2. Реалізувати програмно завдання, які наведені у Додатку 4, згідно варіанту здобувача;

3. Провести тестування кожного із програмно реалізованих завдань.

Контрольні запитання

1. Чи однаково створені змінні типу Symbol є рівні між собою? 2. Чи може undefined використовуватись для присвоєння порожнього значення?

3. Що повертає Boolean(String), Number(Undefined) та String (Null)? 4. Чи працює піднімання (hoisting) для змінних оголошених через let і const? А для var?

5. Чи вірно, що всі змінні які використовуються у функції, мають бути оголошені як локальні?

6. За допомогою чого оголошуються об'єкти/масиви для того, щоб запобігати будь-якій випадковій зміні типу?

Додаток 4. Перелік вимог згідно варіанту до лабораторної роботи 3

№ варіанту	Вимоги

1	1. Створити 10 записів товарів, які містять артикул товару від виробника (літери та цифри), внутрішній id, назву товару, країну походження, вартість1_недіюча, вартість2_недіюча, вартість3_діюча, кількість на складі. Реалізувати програмно: • впорядкування товарів за країною походження та знаходження середньої вартості товару, з однієї і тієї ж країни. • знаходження товару, який має мінімальну «вартість2_недіюча» та вивід внутрішнього id, що йому відповідає. • додавання нового запису до 10 записів: якщо у нього відсутня хоч якась інфо то додаємо в кінець списку товарів; якщо вся інфо наявна, то на початок списку товарів. • створити список, який містить динаміку зміни ціни на кожен товару (за його id або назвою), виходячи з наявних цін, за правилом якщо ціни утворюють спадну (зростаючу) послідовність, то товар «стабільно дешевшає» («стабільно дорожчає»), у протилежному випадку товар «не стабільні зміни». **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести середній вік користувачів і якщо він співпадає із віком деякого користувача, то надати його персональні дані про освіту. • поділити користувачів за часом звернення та метою на три класи – «звернулись з претензією у робочий час» (з 09:00 до 18:00), «звернулись з побажанням у робочий час» та «Інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів у порядку спадання віку та в алфавітному порядку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	**Вимоги**

2	1. Створити 10 записів проєктів, що містять їх внутрішній id, назву проєкту, країну реалізації проєкту, кількість осіб у команді, що його реалізує, тривалість проєкту, вартість проєкту. Реалізувати програмно: • впорядкування проєктів за країною реалізації та знаходження середньої вартості проєкту, який реалізує одна й та ж команда. • знаходження проєкту, який має максимальну кількість осіб у команді, та вивід його внутрішнього id. • додавання нового проєкту до 10 записів: якщо у ньому відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то шукаємо його місце у порядку зростання вартості. • обчислити нову вартість проєкту за принципом: якщо існує принаймні один проєкт, що відрізняється від даного, країни реалізації якого належать одному континенту з більшою кількістю осіб у команді та меншою вартістю, то вартість зростає у 1,4 рази; якщо існує принаймні один проєкт, що відрізняється від даного, країни реалізації якого належать різним континентам з більшою кількістю осіб у команді та меншою вартістю, то вартість зростає у 1,6 разів; якщо існує принаймні один проєкт, що відрізняється від даного, країни реалізації якого належать різним континентам з більшою кількістю осіб у команді та більшою вартістю, то вартість зростає у 2 рази; якщо існує принаймні один проєкт, що відрізняється від даного, країни реалізації якого співпадають та меншою вартістю, то вартість спадає у 0,8 рази; в усіх інших випадках вартість не змінна. Надати оновлений список нових вартостей проєктів – назва проєкту та вартість. **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, кількість символів у полі детально, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести максимальну довжину символів у полі детально і вік користувача, що йому відповідає, та дані про освіту. • поділити користувачів за датою звернення та віком на три класи – «бізнес-активні середнього віку» (вік від 35 до 55 років, місяці звернення - осінь, зима, весна)», «бізнес-пасивні похилого» (літо) та «Інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів алфавітному порядку із виведенням приблизного року народження. **Вказівка:** використовуємо об'єктно-орієнтований js-код. Прізвище та ім'я.

№ варіанту	**Вимоги**

3	1. Створити 10 записів новин, що містять внутрішній id новини, назву новини, автора, кількість символів у короткому повідомлені, кількість користувачів, що прочитали новину за добу_1, кількість користувачів, що прочитали новину за добу_2, час публікації новини вперше (дата, год і хв). Реалізувати програмно: • впорядкування новин за автором та знаходження середньої довжини повідомлення у символах, одного й того ж автора. • знаходження новини, який має максимальну кількість користувачів, що прочитали новину за добу_1 та вивід її внутрішнього id. • додавання нової новини до 10 записів: якщо у ній відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то на початок списку товарів. • обчислити суму оплати роботи автора за добу за принципом: 1 символ у новині, яка з'явилась у робочий час доби (з 10:00 до 18:00) дорівнює 0,5 грн; решта часу – 0,8 грн. **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести мінімальний вік користувача і якщо він співпадає із віком деякого користувача, то надати його дані про освіту, як типові. • поділити користувачів за прізвищем та освітою на три класи – «Жінки з вищою освітою», «Чоловіки без освіти» та «Інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів у порядку спадання віку та в алфавітному порядку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	Вимоги

4	1. Створити 10 записів курсів, що містять внутрішній id , назву, автор, викладачів, тип завдань, кількість користувачів, що прочитали про курс за добу_1, кількість користувачів, що прочитали про курс за добу_2, тривалість вивчення (год, хв). Реалізувати програмно: • впорядкування курсів за тривалістю вивчення та знаходження середньої кількості користувачів, що нею цікавляться. • знаходження курсу, яка має мінімальну кількість користувачів, що прочитали про неї за добу_2 та вивід його внутрішнього id. • додавання нової курсу до 10 записів: якщо у ньому відсутня хоч якась інфо, то додаємо на початок списку; якщо вся інфо є, то шукаємо місце у списку, що впорядкований за автором. • обчислити тривалість вивчення декількох курсів одночасно за принципом: якщо вивчаєш до 3 курсів, то тривалість вивчення кожної з них не змінюється, в іншому випадку тривалість вивчення кожного з них збільшується у 1,5 рази. **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів дітей, які містять прізвище, ім'я, вік, E-mail, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку дітей, батьки яких звернулись продовж певного місяця, в деякий заданий момент часу. • вивести мінімальний вік дитини, E-mail, який їй відповідає, та дату звернення. • поділити дітей за віком на класи – «ранній (до 3-х років включно)», «середній (від 3 до 5)», «дошкільний (5-7 років)» та шкільний (більше 7 років) та знайти кількість дітей у кожному з класів. • забезпечити сортування дітей алфавітному порядку E-mail із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	**Вимоги**

5	1. Створити 10 записів напрямів діяльності, що містять внутрішній id напряму, назву напряму, назву відділу, що його реалізує, текст про зв'язок з державними органами, кількість користувачів, що прочитали про напрям за добу_1, кількість користувачів, що прочитали про напрям за добу_2, терміни реалізації у деяких одиницях (кількість років, місяців, тощо). Реалізувати програмно: • впорядкування напрямів за терміном реалізації та знаходження середньої кількості користувачів, що читають про ті з них, які мають один і той самий термін. • знаходження напряму, який має максимальну кількість користувачів, що прочитали про нього за добу_1 та вивід назви відділу, що його реалізує. • додавання нового напряму до 10 записів: якщо у нього відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то на початок списку. • обчислити тривалість реалізації декількох напрямів одночасно, відповідним йому відділом, за принципом: якщо 1 напрям , то тривалість дорівнює вказаній, додавання напряму у роботу проєкту збільшує його тривалість на 1,1 від вказаної у комірці «терміни реалізації у деяких одиницях (кількість років, місяців, тощо)» . **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести мінімальний вік користувача і та надати його дані про освіту. • поділити користувачів за часом звернення та освітою на три класи – «з вищою освітою у робочий час (09:00 до 17:00)», «без освіти у неробочий час» та «інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів алфавітному порядку із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	**Вимоги**

6	1. Створити 10 записів напрямів діяльності, що містять внутрішній id напряму, назву напряму, назву відділу, що його реалізує, текст про зв'язок з державними органами, кількість користувачів, що прочитали про напрям за добу_1, кількість користувачів, що прочитали про напрям за добу_2, тривалість послуги у деяких одиницях (кількість років, місяців, тощо). Реалізувати програмно: • впорядкування напрямів за тривалістю послуги та знаходження середньої кількості користувачів, що читають про ті з них, що мають одну і ту ж тривалість. • знаходження напряму, який має мінімальну кількість користувачів, що прочитали про нього за добу_2 та вивід назви відділу, що його реалізує. • додавання нового напряму до 10 записів: якщо у нього відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то шукаємо місце у списку за внутрішнім id. • обчислити тривалість надання декількох послуг одночасно, відповідним до нього відділом, за принципом: якщо 1 послуга , то вказану тривалість множимо на 0,95; якщо до 3 послуг, то вказану тривалість множимо на 0,95, в іншому випадку вказану тривалість кожної послуги множимо на 0,6 . **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести середній вік користувача. • поділити користувачів за часом звернення та віком на три класи – «середній за віком, зі зверненням у робочий час (09:00 до 17:00)», «похилого віку зі зверненням у неробочий час» та «інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів алфавітному порядку із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	**Вимоги**

7	1. Створити 10 записів, які є складовими в он-лайн енциклопедії та містять внутрішній id, назву, автора, кількість символів у короткому повідомлені, кількість користувачів, що прочитали за добу_1, кількість користувачів, що прочитали за добу_2, час публікації лоту вперше (дата та год і хв). Реалізувати програмно: • впорядкування за часом публікації та знаходження середньої довжини повідомлення у символах, одного й того часу доби. • знаходження тих складових, які має мінімальну кількість користувачів, що прочитали новину за добу_2 та вивід його внутрішнього id. • додавання нового запису до 10 записів: якщо у ньому відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то шукаємо місце у списку у порядку зростання кількості символів у короткому повідомлені. • обчислити суму оплати роботи автора за принципом: 1 символ у новині, яка з'явилась влітку та взимку дорівнює 0,95 грн, у решта часу – 1,1 грн. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести максимальний вік користувача, що звернувся, його освіту та дату звернення. • поділити користувачів за часом звернення та метою на три класи – «претензійні сови», «доброзичливі жайворонки» та «інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів алфавітному порядку із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	**Вимоги**

8	1. Створити 10 записів послуг, що містять внутрішній id послуги, назву послуги, ім'я лікаря, що її проводить, кількість користувачів, що прочитали про послугу за місяць_1, кількість користувачів, що прочитали про послугу за місяць_1, вартість послуги; тривалість послуги у хв. Реалізувати програмно: • впорядкування послуг за вартістю та знаходження середньої тривалості послуги у хв, серед тих, що відповідають одній і тій же вартості. • знаходження послуги, який має максимальну кількість користувачів, що прочитали про неї за місяць_1 та вивід її внутрішнього id. • додавання нової послуги до 10 записів: якщо у ній відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то шукаємо її місце у порядку спадання вартості послуг. • обчислити оновлену вартість послуги на наступний місяць за принципом: якщо кількість користувачів, що прочитали про послугу за місяць_1 менша за кількість користувачів, що прочитали про послугу за місяць_2, то вартість проєкту множимо на 1,15; якщо кількість користувачів, що прочитали про послугу за місяць__1 дорівнює кількості користувачів, що прочитали про послугу за місяць__2, то вартість проєкту не змінюється; якщо кількість користувачів, що прочитали про послугу за місяць_1 більша за кількість користувачів, що прочитали про послугу за місяць__2, то вартість проєкту множимо на 0,95; Надати оновлений список нових вартостей послуг – назва послуги, вартість, ім'я лікаря\психолога\дизайнера. **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, освіту, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести мінімальний вік користувача, що звернувся, його освіту та ім'я . • поділити користувачів за часом звернення на два класи – «робочий час (09:00 до 17:00)» та «неробочий час» та знайти кількість користувачів у кожному з класів та середній вік користувача у кожному з них. • забезпечити сортування користувачів алфавітному порядку із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	**Вимоги**

9	1. Створити 10 записів про сервіси, що містять внутрішній id товару, його назву, тип сервісу, кількість користувачів, що прочитали про нього за добу_1, кількість користувачів, що прочитали про нього за добу_2, терміни тривалості надання сервісу у деяких одиницях (кількість днів, год, тощо), вартість сервісу. Реалізувати програмно: • впорядкування назв сервісів за терміном тривалості його надання та знаходження середньої кількості користувачів, що читають про них, з одним і тим же терміном (і по добах і по кількості усереднюємо). • знаходження сервісу, який має максимальну кількість користувачів, що прочитали про нього за добу_2 та вивід його типу. • додавання нового запису до 10 записів: якщо у нього відсутня хоч якась інфо, то додаємо в кінець списку; якщо вся інфо є, то шукаємо місце у списку, що відсортований у порядку зростання вартості. • обчислити тривалість надання декількох сервісів одночасно за їх типом: якщо 1 сервіс, то тривалість дорівнює вказаній, якщо їх до 3-х, то тривалість кожного множимо на 1,1 від вказаної початкової, у всіх інших випадках тривалість кожного сервісу множимо на 2. **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, E mail, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись продовж певного місяця, в деякий заданий момент часу. • вивести мінімальний вік користувача, що звернувся, його E-mail та дату звернення. • поділити користувачів на три класи – «активна молодь» (якщо вік до 25, а мета звернення – побажання), «похилий претензійний» (якщо вік більше 60, а мета звернення – претензія),«середній без претензій» (якщо вік менше 60 і більше 25, а мета звернення – не претензія) та «інші». Знайти кількість користувачів у кожному з класів. • забезпечити сортування користувачів алфавітному порядку адреси E-mail із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

№ варіанту	Вимоги

10	1. Створити 10 записів тем, що містять внутрішній id теми, назву теми, автора, тип завдань, кількість користувачів, що прочитали про тему за добу_1, кількість користувачів, що прочитали про тему за добу_2, тривалість вивчення (год, хв). Реалізувати програмно: • впорядкування тем за тривалістю вивчення та знаходження середньої кількості користувачів, що нею цікавляться. • знаходження теми, яка має мінімальну кількість користувачів, що прочитали про неї за добу_2 та вивід її внутрішнього id. • додавання нової теми до 10 записів: якщо у ньому відсутня хоч якась інфо, то додаємо на початок списку; якщо вся інфо є, то шукаємо місце у списку, що впорядкований за автором. • обчислити тривалість вивчення декількох тем одночасно за принципом: якщо вивчаєш до 3 тем, то тривалість вивчення кожної з них не змінюється, в іншому випадку тривалість вивчення кожної з них збільшується у 1,5 рази. **Вказівка:** використовуємо тип даних масив, методи роботи з ними та функції. 2. Створити 10 облікових записів користувачів, які містять прізвище, ім'я, вік, E-mail, мету зворотнього зв'язку, дату звернення, час звернення. Реалізувати програмно: • виведення переліку користувачів, який звернулись у певний місяць і в деякий заданий момент часу. • вивести мінімальний вік користувача, що звернувся, його E-mail та дату звернення. • поділити користувачів за віком та датою звернення на три класи – «школярі до початку навчального року» (вік до 16 років), «не школярі впродовж семестру» та «інші». Кількість користувачів у кожному з класів. • забезпечити сортування користувачів алфавітному порядку E-mail із виведенням мети зворотнього зв'язку. **Вказівка:** використовуємо об'єктно-орієнтований js-код.

СПИСОК ЛІТЕРАТУРИ

1. Мельник Р. Програмування веб-застосувань (фронт-енд та бек-енд). — Львів: Львівська політехніка, 2018. — 250 с.

1. Двірничук К.В., Вацек Д.О. Веб-програмування та веб-дизайн: навч. посіб. — Чернівці : Чернівец. нац. ун-т ім. Ю. Федьковича. — 2022. — 472 с.

2. Duckett J. HTML & CSS: Design and Build Web Sites. — O'Reilly, 2011. — 512 pp. 3. Robson E., Freeman E. Head First HTML and CSS: A Learner's Guide to Creating Standards-Based Web Pages. — O'Reilly, 2012. – 762 pp.

4. HTML Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/html/

5. CSS Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/css/ 6. Responsive web design basics [Електронний ресурс]. Режим доступу: https://web.dev/articles/responsive-web-design-basics

7. Top 10 Front-End Frameworks for Responsive Design 2024 [Електронний ресурс]. Режим доступу: https://daily.dev/blog/top-10-front-end-frameworks-for-responsive-design-2024 8. Robson E., Freeman E. Head First HTML and CSS: A Learner's Guide to Creating Standards-Based Web Pages. — O'Reilly, 2012 – 762 pp.

9. Довідник з підтримки тегів різними браузерами [Електронний ресурс]. Режим доступу: https://www.w3schools.com/tags/ref_html_browsersupport.asp

10. Flanagan D. JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language. — O'Reilly, 2020. — 704 pp.

11. McMillan M. Data Structures and Algorithms with JavaScript: Bringing classic computing approaches to the Web. — O'Reilly, 2014. — 370 pp.

12. JavaScript Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/js/

13. Resources for developers, by developers. MDN Web Docs [Електронний ресурс] – Режим доступу: https://developer.mozilla.org/en-US/docs/Web

14. React Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/js/ 15. Vue Tutorial Електронний ресурс]. Режим доступу: https://vuejs.org/guide/quick start.html

16. Next Tutorial [Електронний ресурс]. Режим доступу: https://nextjs.org/ 17. Angular Tutorial [Електронний ресурс]. Режим доступу: https://angular.dev/ 18. Aluri Y. S., Frontend Performance Optimization of Large-Scale E-commerce Landing Pages: A Comprehensive Analysis // International Journal of Computational and Experimental Science and ENgineering (IJCESEN). — Vol. 11 (4). — 2025. — pp. 9123-9134. 19. Fauzan P., Hasbullah, Farhan M., Reni P. Technical Performance Comparison of Modern Frontend frameworks Study on Svelte, React, and Vue // Brilliance: Research of Artificial Intelligence — Vol. 5 (1). — May 2025.